Database Management: Advanced SQL Concepts (AdventureWorks2022)

This note uses the **AdventureWorks2022** database, a sample SQL Server database modeling a bicycle manufacturing and sales company. All examples are based on standard tables in schemas like `Sales`, `Production`, `HumanResources`, and `Person`.

1. Advanced Queries and Joins

Overview

Advanced queries in SQL Server leverage subqueries, Common Table Expressions (CTEs), and joins to analyze data in AdventureWorks2022. Joins combine rows from tables like `Sales.SalesOrderHeader` and `Person.Person` based on related columns.

Expanded Explanations and Definitions

- **Subqueries**: A nested query executed first to provide results for the outer query. They can be non-correlated (independent) or correlated (dependent on outer query rows).
  - **Sub-explanation**: Subqueries filter or compute values in `WHERE` or `SELECT` clauses.
  - **Code Example**:

```
SELECT FirstName
FROM Person.Person
WHERE BusinessEntityID = (SELECT BusinessEntityID FROM
HumanResources.Employee WHERE NationalIDNumber = '295847284');
```

    - **Comment**: The subquery returns one `BusinessEntityID`, and the outer query fetches the corresponding name, fulfilling the subquery's filtering role.
- **Common Table Expressions (CTEs)**: Temporary result sets defined with `WITH`, improving readability and supporting recursion (e.g., for hierarchical data in `Production.BillOfMaterials`).
  - **Sub-explanation**: CTEs are reusable within a query, unlike subqueries.
  - **Code Example**:

```
WITH ProductCTE AS (
    SELECT ProductID, Name
    FROM Production.Product
)
SELECT Name FROM ProductCTE WHERE ProductID = 1;
```

    - **Comment**: The CTE stores product data, and the outer query retrieves one name, showing CTEs' role in organizing data.
- **Joins**: Combine rows from multiple tables based on a condition.
  - **Types of Joins**:

- **INNER JOIN**: Returns only matching rows from both tables.
  - **Code Example**:

```
SELECT p.FirstName, s.SalesOrderID
FROM Person.Person p
INNER JOIN Sales.SalesOrderHeader s ON p.BusinessEntityID =
s.CustomerID;
```

    - **Comment**: Only customers with orders are included, fulfilling the match-only requirement.
- **LEFT (OUTER) JOIN**: Returns all rows from the left table, with NULLs for non-matching right table rows.
  - **Code Example**:

```
SELECT p.FirstName, s.SalesOrderID
FROM Person.Person p
LEFT JOIN Sales.SalesOrderHeader s ON p.BusinessEntityID =
s.CustomerID;
```

    - **Comment**: All persons appear, with NULLs for those without orders, showing left table inclusion.
- **RIGHT (OUTER) JOIN**: Returns all rows from the right table, with NULLs for non-matching left table rows.
  - **Code Example**:

```
SELECT p.FirstName, s.SalesOrderID
FROM Person.Person p
RIGHT JOIN Sales.SalesOrderHeader s ON p.BusinessEntityID =
s.CustomerID;
```

    - **Comment**: All orders appear, with NULLs for non-existent customers, prioritizing the right table.
- **FULL (OUTER) JOIN**: Returns all rows from both tables, with NULLs for non-matches.
  - **Code Example**:

```
SELECT p.FirstName, s.SalesOrderID
FROM Person.Person p
FULL JOIN Sales.SalesOrderHeader s ON p.BusinessEntityID =
s.CustomerID;
```

    - **Comment**: Includes all persons and orders, with NULLs for non-matches, fulfilling complete inclusion.
- **CROSS JOIN**: Produces a Cartesian product of all rows.

- **Code Example**:

```
SELECT p.Name, c.Name
FROM Production.Product p
CROSS JOIN Production.ProductCategory c;
```

  - **Comment**: Pairs every product with every category, showing all combinations.

- **SELF JOIN**: Joins a table to itself for hierarchical data.
  - **Code Example**:

```
SELECT e1.FirstName AS Employee, e2.FirstName AS Manager
FROM HumanResources.Employee e1
INNER JOIN HumanResources.Employee e2 ON e1.BusinessEntityID
=
e2.OrganizationNode.GetAncestor(1).GetDescendant(e1.Organizat
ionNode, NULL);
```

  - **Comment**: Links employees to managers, showing hierarchical relationships.

Examples

Basic: INNER JOIN

```
SELECT p.FirstName, p.LastName, s.SalesOrderID
FROM Person.Person p
INNER JOIN Sales.SalesOrderHeader s ON p.BusinessEntityID = s.CustomerID
WHERE s.OrderDate >= '2014-01-01';
```

Explanation: Retrieves customer names and order IDs for orders after January 1, 2014.

Intermediate: Correlated Subquery

```
SELECT soh.SalesOrderID, soh.TotalDue
FROM Sales.SalesOrderHeader soh
WHERE soh.TotalDue > (
    SELECT AVG(TotalDue)
    FROM Sales.SalesOrderHeader
    WHERE SalesPersonID = soh.SalesPersonID
);
```

Explanation: Finds orders with totals above the average for their salesperson.

Advanced: Recursive CTE for Bill of Materials

```
WITH BOM AS (
    SELECT ProductAssemblyID, ComponentID, PerAssemblyQty, 0 AS Level
    FROM Production.BillOfMaterials
    WHERE ProductAssemblyID = 800  -- Example: ML Road Frame
    UNION ALL
    SELECT b.ProductAssemblyID, b.ComponentID, b.PerAssemblyQty, Level + 1
    FROM Production.BillOfMaterials b
    INNER JOIN BOM ON b.ProductAssemblyID = BOM.ComponentID
)
SELECT p.Name, BOM.PerAssemblyQty, BOM.Level
FROM BOM
INNER JOIN Production.Product p ON BOM.ComponentID = p.ProductID;
```

Explanation: Builds a hierarchical bill of materials for product ID 800, showing components and levels.

Real-Life Use Case

Sales Reporting: AdventureWorks joins `Sales.SalesOrderHeader`, `Sales.SalesOrderDetail`, and `Production.Product` to identify top-selling products, using subqueries to filter high-value orders, informing marketing strategies.

---

2. Views, Stored Procedures, and Query Metadata

Views

Overview

Views are virtual tables defined by SQL queries, simplifying data access in AdventureWorks2022's schemas like `Sales` and `Person`.

Expanded Explanations and Definitions

- **View**: A saved query acting like a table, without storing data physically.
  - **Sub-explanation**: Abstracts complex queries for usability.
  - **Code Example**:

    ```
    CREATE VIEW Sales.vCustomerNames AS
    SELECT FirstName, LastName
    FROM Person.Person;
    ```

    - **Comment**: Hides other columns, showing only names, fulfilling data abstraction.
- **Updatable View**: Allows modifications if based on one table with no aggregations. `WITH CHECK OPTION` enforces the view's `WHERE` clause.
  - **Sub-explanation**: Ensures updates align with the view's scope.

    - **Code Example**:

```
CREATE VIEW HumanResources.vEmployeePay AS
SELECT BusinessEntityID, Rate
FROM HumanResources.EmployeePayHistory
WITH CHECK OPTION;
```

        - **Comment**: Allows rate updates, ensuring compliance, fulfilling updatability.
- **Materialized View**: A physical data copy (SQL Server uses indexed views).
    - **Sub-explanation**: Stores data for performance.
    - **Code Example**:

```
CREATE VIEW Sales.vSalesSummary WITH SCHEMABINDING AS
SELECT SalesPersonID, COUNT_BIG(*) AS OrderCount
FROM Sales.SalesOrderHeader
GROUP BY SalesPersonID;
```

        - **Comment**: Enables indexing, showing materialized view functionality.

Examples

Basic: Simple View

```
CREATE VIEW Sales.vActiveCustomers AS
SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE BusinessEntityID IN (SELECT CustomerID FROM Sales.SalesOrderHeader);
```

Explanation: Shows customers with sales orders.

Intermediate: View with Joins

```
CREATE VIEW Sales.vOrderSummary AS
SELECT soh.SalesOrderID, soh.OrderDate, p.FirstName, p.LastName
FROM Sales.SalesOrderHeader soh
INNER JOIN Person.Person p ON soh.CustomerID = p.BusinessEntityID
WHERE soh.Status = 5;  -- Completed orders
```

Explanation: Combines order and customer data for completed orders.

Advanced: Updatable View

```
CREATE VIEW Person.vEmployeeContact AS
SELECT BusinessEntityID, EmailAddress
FROM Person.EmailAddress
WHERE BusinessEntityID IN (SELECT BusinessEntityID FROM HumanResources.Employee)
WITH CHECK OPTION;
```

Explanation: Allows updates to employee email addresses, restricted to employees.

Real-Life Use Case

Customer Service: AdventureWorks creates a view joining `Sales.Customer`, `Person.Person`, and `Sales.SalesOrderHeader` to display order histories for support agents, excluding sensitive data like credit card numbers.

Stored Procedures

Overview

Stored procedures are precompiled SQL scripts in AdventureWorks2022, executed with parameters to encapsulate logic.

Expanded Explanations and Definitions

- **Stored Procedure**: A named set of SQL statements, executed server-side.
  - **Sub-explanation**: Reduces network traffic and ensures consistency.
  - **Code Example**:

```
CREATE PROCEDURE Sales.uspGetCustomerName
    @CustomerID INT
AS
BEGIN
    SELECT FirstName, LastName
    FROM Person.Person
    WHERE BusinessEntityID = @CustomerID;
END;
```

    - **Comment**: Retrieves a customer's name, showing server-side execution.
- **Parameters**: `IN`, `OUT`, or `INOUT` for dynamic inputs/outputs.
  - **Sub-explanation**: Enable reusable logic.
  - **Code Example**:

```
CREATE PROCEDURE Sales.uspUpdateOrderStatus
    @OrderID INT,
    @NewStatus TINYINT
AS
BEGIN
    UPDATE Sales.SalesOrderHeader
```

```
    SET Status = @NewStatus
    WHERE SalesOrderID = @OrderID;
END;
```

    - **Comment**: Updates status using parameters, fulfilling dynamic handling.
- **Dynamic SQL**: Builds and executes SQL at runtime.
    - **Sub-explanation**: Offers flexibility with sanitization to prevent injection.
    - **Code Example**:

```
CREATE PROCEDURE Sales.uspDynamicQuery
    @ColumnName NVARCHAR(50)
AS
BEGIN
    DECLARE @SQL NVARCHAR(200) = N'SELECT ' + QUOTENAME(@ColumnName) +
' FROM Sales.SalesOrderHeader';
    EXECUTE sp_executesql @SQL;
END;
```

    - **Comment**: Executes a dynamic query, showing flexibility.

Examples

Basic: Simple Update

```
CREATE PROCEDURE Sales.uspUpdateOrderDate
    @SalesOrderID INT,
    @NewDate DATE
AS
BEGIN
    UPDATE Sales.SalesOrderHeader
    SET OrderDate = @NewDate
    WHERE SalesOrderID = @SalesOrderID;
END;
EXEC Sales.uspUpdateOrderDate @SalesOrderID = 43659, @NewDate = '2014-07-01';
```

Explanation: Updates the order date for a specific sales order.

Intermediate: Conditional Logic

```
CREATE PROCEDURE Sales.uspApplyDiscount
    @SalesOrderID INT,
    @DiscountPercent DECIMAL(5,2)
AS
BEGIN
    DECLARE @TotalDue DECIMAL(10,2);
    SELECT @TotalDue = TotalDue FROM Sales.SalesOrderHeader WHERE SalesOrderID =
```

```
@SalesOrderID;
    IF @TotalDue > 5000
    BEGIN
        UPDATE Sales.SalesOrderHeader
        SET TotalDue = TotalDue * (1 - @DiscountPercent / 100)
        WHERE SalesOrderID = @SalesOrderID;
    END;
END;
EXEC Sales.uspApplyDiscount @SalesOrderID = 43659, @DiscountPercent = 10.00;
```

Explanation: Applies a discount to orders over $5,000.

Advanced: Dynamic SQL

```
CREATE PROCEDURE Sales.uspGenerateSalesReport
    @GroupColumn NVARCHAR(50)
AS
BEGIN
    DECLARE @SQL NVARCHAR(500);
    SET @SQL = N'SELECT ' + QUOTENAME(@GroupColumn) + ', SUM(TotalDue) AS
TotalSales
                FROM Sales.SalesOrderHeader
                GROUP BY ' + QUOTENAME(@GroupColumn);
    EXECUTE sp_executesql @SQL;
END;
EXEC Sales.uspGenerateSalesReport @GroupColumn = 'SalesPersonID';
```

Explanation: Generates a sales report grouped by a specified column.

Real-Life Use Case

Order Processing: AdventureWorks uses a stored procedure to update `Sales.SalesOrderHeader`, `Sales.SalesOrderDetail`, and `Production.ProductInventory` in a transaction, ensuring consistent stock and order status.

Query Metadata

Overview

Metadata queries retrieve AdventureWorks2022's structure (e.g., tables, columns) using `INFORMATION_SCHEMA` or `sys` catalogs.

Expanded Explanations and Definitions

- **Metadata**: Data about the database schema, stored in system tables.
  - **Sub-explanation**: Supports documentation and automation.
  - **Code Example**:

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'Sales';
```

    - **Comment**: Lists `Sales` schema tables, fulfilling schema discovery.
- **System Catalogs**: Tables/views like `INFORMATION_SCHEMA` or `sys` schemas.
    - **Sub-explanation**: Provide metadata access.
    - **Code Example**:

```
SELECT name FROM sys.tables WHERE schema_name(schema_id) =
'Production';
```

        - **Comment**: Lists `Production` tables, showing native metadata access.
- **Constraints**: Rules like primary keys or foreign keys.
    - **Sub-explanation**: Ensure data integrity.
    - **Code Example**:

```
SELECT CONSTRAINT_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE TABLE_NAME = 'SalesOrderHeader' AND CONSTRAINT_TYPE = 'PRIMARY
KEY';
```

        - **Comment**: Identifies primary key, fulfilling constraint retrieval.

Examples

Basic: List Tables

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'Production';
```

Explanation: Lists tables in the `Production` schema.

Intermediate: Column Details

```
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Product';
```

Explanation: Retrieves column metadata for the `Production.Product` table.

Advanced: Foreign Key Relationships

```
SELECT tc.CONSTRAINT_NAME, kcu.COLUMN_NAME, ccu.TABLE_NAME AS ReferencedTable
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE kcu ON tc.CONSTRAINT_NAME =
kcu.CONSTRAINT_NAME
JOIN INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE ccu ON tc.CONSTRAINT_NAME =
ccu.CONSTRAINT_NAME
WHERE tc.CONSTRAINT_TYPE = 'FOREIGN KEY' AND tc.TABLE_NAME = 'SalesOrderDetail';
```

Explanation: Lists foreign keys in `Sales.SalesOrderDetail`, including referenced tables.

Real-Life Use Case

Compliance Auditing: AdventureWorks queries metadata to document `Sales` and `Production` schemas, listing tables and constraints for regulatory audits.

---

3. Indexes

Overview

Indexes in AdventureWorks2022 enhance query performance on tables like `Production.Product`, trading faster reads for slower writes.

Expanded Explanations and Definitions

- **Index**: A data structure (e.g., B-tree) storing column values and row pointers.
  - **Sub-explanation**: Reduces table scans.
  - **Code Example**:

    ```
    CREATE INDEX IX_Product_Name ON Production.Product(Name);
    ```

    - **Comment**: Speeds up name searches, fulfilling performance enhancement.
- **Primary Index**: Created for primary keys.
  - **Sub-explanation**: Ensures uniqueness and fast lookups.
  - **Code Example**:

    ```
    SELECT * FROM sys.indexes WHERE object_id =
    OBJECT_ID('Sales.SalesOrderHeader') AND is_primary_key = 1;
    ```

    - **Comment**: Shows primary index, fulfilling automatic creation.
- **Clustered Index**: Determines physical data order (one per table).
  - **Sub-explanation**: Stores data in index order.
  - **Code Example**:

```
CREATE CLUSTERED INDEX IX_SalesOrderHeader_SalesOrderID ON
Sales.SalesOrderHeader(SalesOrderID);
```

    - **Comment**: Orders data by `SalesOrderID`, fulfilling physical ordering.
- **Non-Clustered Index**: Stores pointers, separate from data.
    - **Sub-explanation**: Optimizes queries flexibly.
    - **Code Example**:

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_CustomerID ON
Sales.SalesOrderHeader(CustomerID);
```

    - **Comment**: Speeds customer queries, showing flexibility.
- **Composite Index**: Indexes multiple columns.
    - **Sub-explanation**: Optimizes multi-column conditions.
    - **Code Example**:

```
CREATE INDEX IX_SalesOrderDetail_ProductID_OrderQty ON
Sales.SalesOrderDetail(ProductID, OrderQty);
```

    - **Comment**: Enhances multi-column queries, fulfilling composite indexing.

Examples

Basic: Single-Column Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate ON
Sales.SalesOrderHeader(OrderDate);
```

Explanation: Speeds up queries filtering by `OrderDate`.

Intermediate: Composite Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ProductID_UnitPrice ON
Sales.SalesOrderDetail(ProductID, UnitPrice);
```

Explanation: Optimizes queries like `WHERE ProductID = 707 AND UnitPrice > 100`.

Advanced: Filtered Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OnlineOrders ON
Sales.SalesOrderHeader(SalesOrderID)
WHERE OnlineOrderFlag = 1;
```

Explanation: Indexes online orders, reducing index size.

Real-Life Use Case

Product Catalog: AdventureWorks indexes `Production.Product (ProductSubcategoryID, ListPrice)` to speed up category and price-based searches, improving online store performance.

---

4. Triggers

Overview

Triggers in AdventureWorks2022 execute automatically in response to DML events on tables like `Sales.SalesOrderDetail`.

Expanded Explanations and Definitions

- **Trigger**: A procedure tied to a table, activated by events.
  - **Sub-explanation**: Enforces rules or logs actions.
  - **Code Example**:

```
CREATE TRIGGER Sales.trgLogOrderInsert
ON Sales.SalesOrderHeader
AFTER INSERT
AS
BEGIN
    INSERT INTO Sales.SalesOrderHeader (SalesOrderID, OrderDate,
Comment)
    SELECT SalesOrderID, GETDATE(), 'Logged via trigger'
    FROM inserted;
END;
```

    - **Comment**: Logs new orders, fulfilling automation (note: this is a simplified example; typically, logging would use a separate table, but we're constrained to existing tables).
- **BEFORE Trigger**: Executes pre-event (SQL Server uses `INSTEAD OF`).
  - **Sub-explanation**: Validates data.
  - **Code Example**:

```
CREATE TRIGGER Sales.trgCheckOrder
ON Sales.SalesOrderHeader
INSTEAD OF INSERT
AS
BEGIN
```

```
    IF EXISTS (SELECT 1 FROM inserted WHERE TotalDue < 0)
        THROW 50001, 'TotalDue cannot be negative.', 1;
    ELSE
        INSERT INTO Sales.SalesOrderHeader SELECT * FROM inserted;
END;
```

        - **Comment**: Prevents negative `TotalDue`, fulfilling validation.
- **AFTER Trigger**: Executes post-event.
    - **Sub-explanation**: Logs changes.
    - **Code Example**:

```
CREATE TRIGGER Sales.trgLogOrderUpdate
ON Sales.SalesOrderHeader
AFTER UPDATE
AS
BEGIN
    INSERT INTO Sales.SalesOrderHeader (SalesOrderID, OrderDate,
Comment)
    SELECT SalesOrderID, GETDATE(), 'Update logged'
    FROM inserted;
END;
```

        - **Comment**: Logs updates, showing post-event action (simplified to use existing table).

Examples

Basic: Update Trigger

```
CREATE TRIGGER Sales.trgUpdateOrderComment
ON Sales.SalesOrderHeader
AFTER INSERT
AS
BEGIN
    UPDATE Sales.SalesOrderHeader
    SET Comment = 'New order added'
    WHERE SalesOrderID IN (SELECT SalesOrderID FROM inserted);
END;
```

Explanation: Adds a comment to new sales orders.

Intermediate: Validation Trigger

```
CREATE TRIGGER Production.trgPreventNegativeInventory
ON Production.ProductInventory
AFTER UPDATE
AS
```

```
BEGIN
    IF EXISTS (SELECT 1 FROM inserted WHERE Quantity < 0)
        THROW 50001, 'Inventory quantity cannot be negative.', 1;
END;
```

Explanation: Prevents negative inventory updates.

Advanced: Cascading Trigger

```
CREATE TRIGGER Sales.trgUpdateOrderSubTotal
ON Sales.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    UPDATE Sales.SalesOrderHeader
    SET SubTotal = (
        SELECT SUM(UnitPrice * OrderQty)
        FROM Sales.SalesOrderDetail sod
        WHERE sod.SalesOrderID = Sales.SalesOrderHeader.SalesOrderID
    )
    WHERE SalesOrderID IN (SELECT SalesOrderID FROM inserted);
END;
```

Explanation: Recalculates `SubTotal` in `SalesOrderHeader` when order details change.

Real-Life Use Case

Inventory Management: AdventureWorks uses triggers on `Production.ProductInventory` to validate stock updates, preventing negative quantities and ensuring accurate supply chain data.

Summary

- **Advanced Queries and Joins**: Analyze AdventureWorks2022 data with subqueries, CTEs, and joins.
- **Views**: Simplify and secure data access.
- **Stored Procedures**: Encapsulate order processing logic.
- **Query Metadata**: Document schemas for audits.
- **Indexes**: Optimize sales and product queries.
- **Triggers**: Automate inventory validation.

Additional Resources

- SQL Server documentation: https://docs.microsoft.com/sql
- AdventureWorks2022: https://github.com/Microsoft/sql-server-samples
- Practice: SQLZoo, LeetCode
- Books: "T-SQL Fundamentals" by Itzik Ben-Gan